

Ejercicio 1: Algo huele mal

Indique qué malos olores se presentan en los siguientes ejemplos.

1.1 Protocolo de Cliente

La clase Cliente tiene el siguiente protocolo. ¿Cómo puede mejorarlo?

```
/**
 * Retorna el límite de crédito del cliente
 */
public double lmtCrdt() {...

/**
 * Retorna el monto facturado al cliente desde la fecha f1 a la fecha f2
 */
protected double mtFcE(LocalDate f1, LocalDate f2) {...

/**
 * Retorna el monto cobrado al cliente desde la fecha f1 a la fecha f2
 */
private double mtCbE(LocalDate f1, LocalDate f2) {...
```

Malos olores:

Nombres de **métodos** y **variables** pocos claros

Solución:

Renombrar métodos y variables

→

```
/**
 * Retorna el límite de crédito del cliente
 */
public double limiteCredito() {...

/**
 * Retorna el monto facturado al cliente desde la fecha f1 a la fecha f2
 */
protected double montoFacturadoEnPeriodo(LocalDate fechaInicio, LocalDate fechaFin)
{...

/**
 * Retorna el monto cobrado al cliente desde la fecha f1 a la fecha f2
 */
private double montoCobradoEnPeriodo(LocalDate fechaInicio, LocalDate fechaFin)
{...
```

1.2 Participación en proyectos

Al revisar el siguiente diseño inicial (Figura 1), se decidió realizar un cambio para evitar lo que se consideraba un mal olor. El diseño modificado se muestra en la Figura 2. Indique qué tipo de cambio se realizó y si lo considera apropiado. Justifique su respuesta.

Diseño inicial:

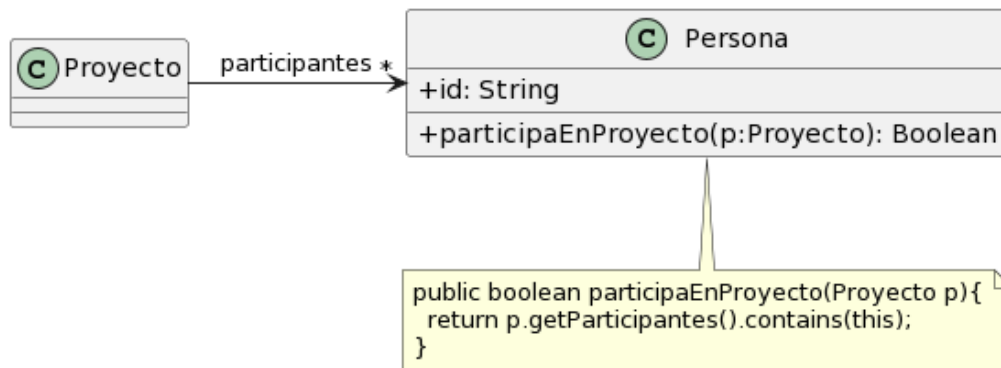


Figura 1: Diagrama de clases del diseño inicial.

Diseño revisado:

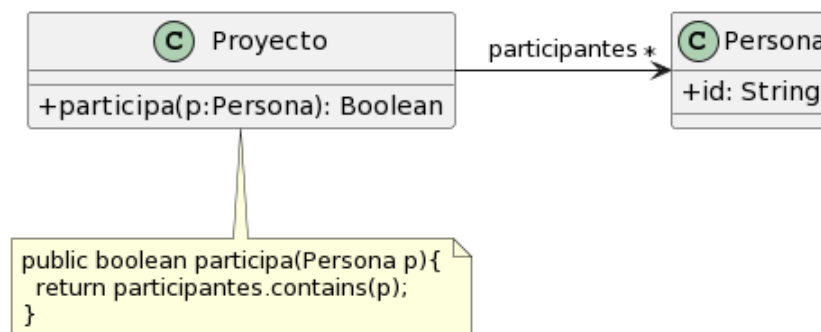


Figura 2: Diagrama de clases modificado.

Respuesta:

Se producía una envidia de atributos entre clases, se eliminó el método de la clase Persona y se creó uno con la misma semántica en la clase Proyecto. Está justificado porque Proyecto ya conoce todos los datos necesarios para realizar la misma consulta sin recurrir a otra.

Faltaría arreglar la privacidad de id y se podría nombrar mejor al método participa

1.3 Cálculos

Analice el código que se muestra a continuación. Indique qué *code smells* encuentra y cómo pueden corregirse.

```

public void imprimirValores() {
    int totalEdades = 0;
    double promedioEdades = 0;
    double totalSalarios = 0;

    for (Empleado empleado : personal) {
        totalEdades = totalEdades + empleado.getEdad();
        totalSalarios = totalSalarios + empleado.getSalario();
    }
    promedioEdades = totalEdades / personal.size();

    String message = String.format("El promedio de las edades es %s y el total de salarios es %s", promedioEdades, totalSalarios);

    System.out.println(message);
}

```

Bad smells:

- Debería inicializar las variables totalEdades, promedioEdades y totalSalarios?
- Nombre poco claro del método
- El for se hace 2 cosas, totalEdad y totalSalarios
- Se usa un foreach con una lista en vez de usar directamente Stream

Posible solución:

- Pasar a métodos separados, el cálculo de la edad total/promedio de edades y total salarios
- Buscar un nombre de método más explícito
- Utilizar Stream

```

public void imprimirEdadPromedioYTotalSalario() {

    String message = String.format("El promedio de las edades es %s y el total de salarios es %s", calcularPromedioEdad(), calcularTotalSalarios());

    System.out.println(message);
}

private double calcularPromedioEdad() {
    return personal.stream()
        .mapToInt(persona -> persona.getEdad())
        .Average().orElse(0);
}

private double calcularTotalSalarios() {
    return personal.stream()
        .mapToDouble(persona -> persona.getSalario())
        .Sum();
}

```

Ejercicio 2

Para cada una de las siguientes situaciones, realice en forma iterativa los siguientes pasos:

- (i) indique el mal olor,
 - (ii) indique el refactoring que lo corrige,
 - (iii) aplique el refactoring, mostrando el resultado final (código y/o diseño según corresponda).
- Si vuelve a encontrar un mal olor, retorne al paso (i).

2.1 Empleados

```
public class EmpleadoTemporario {
    public String nombre;
    public String apellido;
    public double sueldoBasico = 0;
    public double horasTrabajadas = 0;
    public int cantidadHijos = 0;
    // .....

    public double sueldo() {
        return this.sueldoBasico
            + (this.horasTrabajadas * 500)
            + (this.cantidadHijos * 1000)
            - (this.sueldoBasico * 0.13);
    }
}

public class EmpleadoPlanta {
    public String nombre;
    public String apellido;
    public double sueldoBasico = 0;
    public int cantidadHijos = 0;
    // .....

    public double sueldo() {
        return this.sueldoBasico
            + (this.cantidadHijos * 2000)
            - (this.sueldoBasico * 0.13);
    }
}

public class EmpleadoPasante {
    public String nombre;
    public String apellido;
    public double sueldoBasico = 0;
    // .....

    public double sueldo() {
        return this.sueldoBasico - (this.sueldoBasico * 0.13);
    }
}
```

i)

Variables de instancia publicas

Variables repetidas + método similar

muchos cálculos en un solo método

codigo en método sueldo, poco claro (los números que se usan en las cuentas que son??)

ii)

Crear una super clases abstract (Extract Superclass)

Arreglar privacidad de variables

Nombre de método más claros

Subir el método compartido a super clase(Pull Up Method)

Extract metod para bajar la carga del metodo

pasar los valores 500, 1000, 2000, 0.13 a variables constantes (debería saber que son)

iii)

```
public abstract class Empleado {
    private String nombre;
    private String apellido;
    protected double sueldoBasico;
    protected double aporteJubilatorio = 0.13;
    // .....

    public double calcularDescuentoAporte() {
        return this.sueldoBasico * this.AporteJubilatorio;
    }

    public abstract double sueldoNeto();
}
```

}

```
public abstract class EmpleadoConHijo extends Empleado {
    protected int cantidadHijos;
    protected int bonoHijos;

    public double calcularExtraPorHijo() {
        return this.cantidadHijos * this.bonoHijos;
    }
}
```

}

```
public class EmpleadoTemporario extends EmpleadoConHijo{
    private double horasTrabajadas = 0;
    private int bonoHoras = 500;

    // .....
```

```

    public double sueldoNeto() {
        return this.sueldoBasico
            + this.calcularExtraPorHoras()
            + this.calcularExtraPorHijo()
            - (this.calcularDescuentoAporte());
    }

    private double calcularExtraPorHoras() {
        return this.horasTrabajadas * this.bonoHoras;
    }
}

```

```

public class EmpleadoPlanta extends EmpleadoConHijo{

    // .....

    public double sueldoNeto() {
        return this.sueldoBasico
            + this.calcularExtraPorHijo()
            - (this.calcularDescuentoAporte());
    }
}

```

```

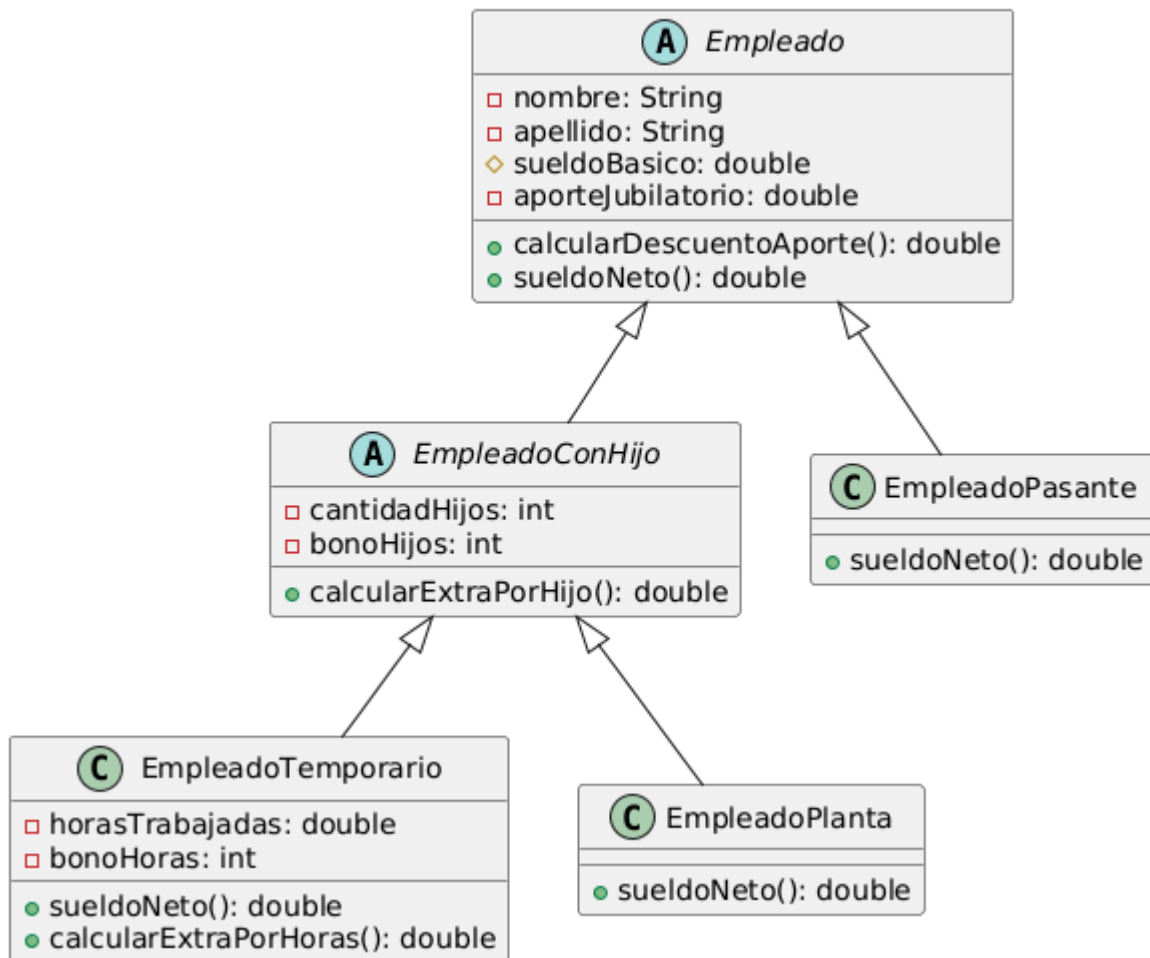
public class EmpleadoPasante extends Empleado{

    // .....

    public double sueldoNeto() {
        return this.sueldoBasico - (this.calcularDescuentoAporte());
    }
}

```

Empty Diagram - a



PREGUNTAR SÍ EL THIS.SUELDOBASICO ESTA BIEN O USO GETS

TAMBIEN EN VEZ DE EmpleadoConHijo SIMPLEMENTEMENTE HEREDAR DE EmpleadoPlanta ES MEJOR?

EmpleadoPasante → EmpleadoPlanta → EmpleadoTemporario? Creo que funcionaria, pero no se si tiene sentido que un EmpleadoPlanta sea un EmpleadoPasante

2.2 Juego

```
public class Juego {  
    // .....  
    public void incrementar(Jugador j) {  
        j.puntuacion = j.puntuacion + 100;  
    }  
    public void decrementar(Jugador j) {  
        j.puntuacion = j.puntuacion - 50;  
    }  
}
```

i)

Nombres de metodos

Nombre de variables

Privacidad de variables

Constantes no declaradas(+100, -50)

Se rompe encapsulamiento

Clase dentro de otra clase?

ii)

Que falten constructores importa?

Rename method

Rename Variables(?)

Encapsulate Field

Move Metod o Extract Metod??? (Paso del metodo a la otra clase)

Rename Method

Extract Class

iii)

```
public class Juego {  
    private int puntosGanados = 100;  
    private int puntosPerdidos = 50;  
  
    public void aumentarPuntuacion(Jugador jugador) {  
        jugador.ganoPuntos(puntosGanados);  
    }  
  
    public void reducirPuntuacion(Jugador jugador) {  
        jugador.pierdoPuntos(puntosPerdidos);  
    }  
}
```

```
public class Jugador {  
    private String nombre;  
    private String apellido;
```

```
public class Jugador {  
    public String nombre;  
    public String apellido;  
    public int puntuacion = 0;  
}
```



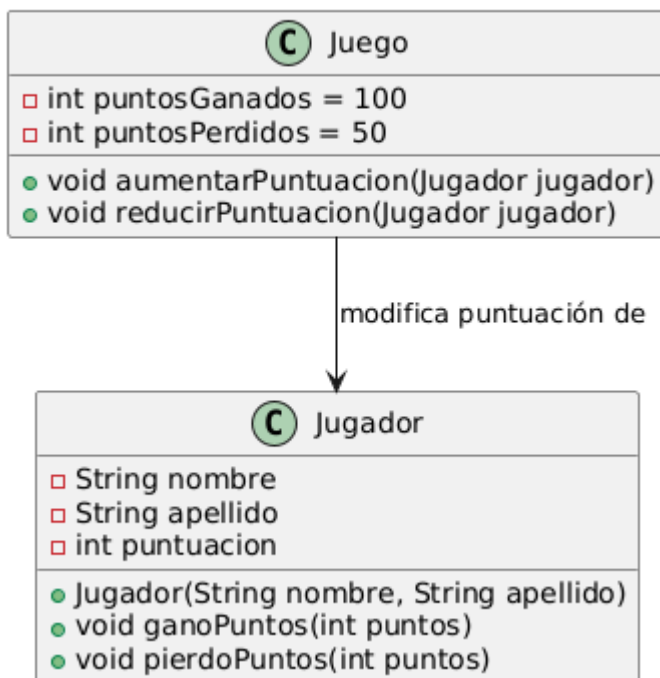
```
private int puntuacion;
```

```
public Jugador(String nombre, String apellido) {  
    this.nombre = nombre;  
    this.apellido = apellido;  
    this.puntuacion = 0;  
}
```

```
public void ganoPuntos(int puntos) {  
    this.puntuacion += puntos;  
}
```

```
public void pierdoPuntos(int puntos) {  
    this.puntuacion -= puntos;  
}  
}
```

Empty Diagram - ejer JuegoJugador



2.3 Publicaciones



```
/**
 * Retorna los últimos N posts que no pertenecen al usuario user
 */
public List<Post> ultimosPosts(Usuario user, int cantidad) {

    List<Post> postsOtrosUsuarios = new ArrayList<Post>();
    for (Post post : this.posts) {
        if (!post.getUsuario().equals(user)) {
            postsOtrosUsuarios.add(post);
        }
    }

    // ordena los posts por fecha
    for (int i = 0; i < postsOtrosUsuarios.size(); i++) {
        int masNuevo = i;
        for (int j = i + 1; j < postsOtrosUsuarios.size(); j++) {
            if (postsOtrosUsuarios.get(j).getFecha().isAfter(
                postsOtrosUsuarios.get(masNuevo).getFecha())) {
                masNuevo = j;
            }
        }
        Post unPost = postsOtrosUsuarios.set(i, postsOtrosUsuarios.get(masNuevo));
        postsOtrosUsuarios.set(masNuevo, unPost);
    }

    List<Post> ultimosPosts = new ArrayList<Post>();
    int index = 0;
    Iterator<Post> postIterator = postsOtrosUsuarios.iterator();
    while (postIterator.hasNext() && index < cantidad) {
        ultimosPosts.add(postIterator.next());
    }
    return ultimosPosts;
}
```

